Reviewer Pack — Minimal Rank of Grothendieck-Witt Class Modulo 2 vs Resoluti…

Entry db0c911f4716 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 03:02:31 UTC

Minimal Rank of Grothendieck-Witt Class Modulo 2 vs Resolution Proof Width

Entry ID: `db0c911f4716` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 03:02:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Grothendieck-Witt Theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> {'statement_1': 'For every satisfiable CNF formula F with n variables, let W(F) be the Grothendieck-Witt class of its clause-indicator polynomial modulo 2.', 'statement_2': 'Then, for all instances with n ≤ 40, the minimal rank of W(F), denoted by rk(W(F)), is linearly proportional to the resolution proof width t*(F).', 'statement_3': 'Equivalently, there exist positive constants α and β such that α t*(F) ≤ rk(W(F)) ≤ β t*(F).'}

Rationale (proposer's reasoning):

> {'rationale_1': 'Grothendieck-Witt theory provides a bridge between the algebraic structure of polynomials and their geometric interpretation, which might reveal hidden structures in resolution proofs.', 'rationale_2': 'The Grothendieck-Witt class is computable for clauses of CNF formulas via the computation of the clause-indicator polynomial modulo 2, making it feasible to test on small instances.', 'rationale_3': 'This conjecture, if true, would offer a new perspective on the complexity of resolution proofs and potentially lead to improved proof complexity lower bounds.'}

Taxonomy category: `Geometric Interpretation` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6dbeaf3b00bd95aa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF formula F with n ≤ 40, if the correlation coefficient between rk(W(F)) and t*(F) is within ±0.1 of 1, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Grothendieck-Witt class" AND "resolution proof complexity" - "minimal rank of Grothendieck-Witt class" MODULO 2 AND resolution proof width" - "linear relationship" minimal rank Grothendieck-Witt class MODULO 2 resolution proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def random_cnf(n, m):
    cnf = []
    for _ in range(m):
        literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
        clause = ' '.join(map(str, literals)) + ' 0'
        cnf.append(clause)
    return '\n'.join(cnf)

def resolution_width(cnf):
    clauses = cnf.split('\n')
    literals = set()
    for clause in clauses:
        literals.update(abs(int(lit)) for lit in clause.split() if lit != '0')

    def dpll(clauses, assignment):
        if not clauses:
            return True
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if unit_clause:
            literal = int(unit_clause[0])
            if literal < 0 and literal in assignment or literal > 0 and -literal in assignment:
                return False
            assignment.add(literal if literal > 0 else -literal)
            clauses = [c for c in clauses if literal not in c and -literal not in c]
        pure_literal = next((l for l in literals if (l in assignment or -l in assignment) == False
        if pure_literal:
            assignment.add(pure_literal)
```

```
            clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
        return dpll(clauses, assignment)

    max_width = 0
    for _ in range(10):  # Repeat DPLL multiple times to get a better estimate of the width
        assignment = set()
        width = 0
        while True:
            if dpll(clauses, assignment):
                break
            new_literal = random.choice(list(literals - assignment))
            assignment.add(new_literal)
            width += 1
        max_width = max(max_width, width)
    return max_width

def grothendieck_witt_class(cnf):
    n = int(cnf.split('\n')[0].split()[2])
    clause_indicator_poly = [0] * (1 << n)
    for clause in cnf.split('\n'):
        if ' 0' not in clause:
            literals = list(map(int, clause.split()))
            product = 1
            for lit in literals:
                if lit > 0:
                    product *= (-1) ** (lit - 1)
                else:
                    product *= (-1) ** (-lit - 1)
            clause_indicator_poly[sum(abs(lit) for lit in literals)] += product

    def matrix_mult(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(p):
                    result[i][j] += A[i][k] * B[k][j]
        return result

    def gaussian_elimination(A, b):
        m = len(A)
        n = len(A[0])
        augmented_matrix = [A[i] + [b[i]] for i in range(m)]
        row, col = 0, 0
        while row < m and col < n:
            if A[row][col] == 0:
                swap_row = next((i for i in range(row + 1, m) if augmented_matrix[i][col]), None)
                if swap_row is None:
                    col += 1
                else:
                    augmented_matrix[row], augmented_matrix[swap_row] = augmented_matrix[swap_row]
            else:
                pivot = augmented_matrix[row][col]
                for j in range(col, n + 1):
```

```
                    augmented_matrix[row][j] /= pivot
                for i in range(m):
                    if i != row and augmented_matrix[i][col] != 0:
                        factor = augmented_matrix[i][col]
                        for j in range(col, n + 1):
                            augmented_matrix[i][j] -= factor * augmented_matrix[row][j]
                row += 1
                col += 1

        rank = sum(1 for i in range(m) if any(augmented_matrix[i][j] != 0 for j in range(n)))
        return rank

    A = []
    b = []
    for i in range(1, len(clause_indicator_poly)):
        A.append([clause_indicator_poly[j] for j in range(i)])
        b.append(clause_indicator_poly[i])

    return gaussian_elimination(A, b)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice(range(5, 41))
    m = int(n * (n - 1) / 2)  # Typical number of clauses for a satisfiable instance
    cnf = random_cnf(n, m)

    try:
        t_star = resolution_width(cnf)
        rk_W_F = grothendieck_witt_class(cnf)

        if t_star == 0 or rk_W_F == 0:
            return {
                "metric_name": "correlation",
                "metric_value": None,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": "t_star or rk_W_F is zero"
            }

        correlation = abs(rk_W_F / t_star)
        return {
            "metric_name": "correlation",
            "metric_value": correlation,
            "instances_tested": 1,
            "conjecture_holds": 0.9 <= correlation <= 1.1,
            "counterexample": ""
        }
    except Exception as e:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": str(e)
        }
```

```python
if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1:
        seeds = list(map(int, sys.argv[1:]))
    else:
        seeds = [2**i + 1 for i in range(5, 6)]  # Default to 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results if r['metric_value'
    elif any("counterexample" in r and r["conjecture_holds"] == False for r in results):
        counterexample = next(r["counterexample"] for r in results if "counterexample" in r and r[
        first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["conje
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_holds': False, 'counterexample': "'in <string>' requires string as left operand, not int"}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f79e23f8.py", line 179, in <module>
    first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["conjecture_hol
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/researc
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing any data, which prevents us from evaluating the conjecture's validity. | next: Review and debug the test code to ensure it can run successfully and produce results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14690
2	preregistration	ollama_remote	glm4:latest	0	0	5368
3	novelty	ollama_remote	glm4:latest	0	0	4829
4	novelty	ollama_remote	glm4:latest	0	0	5767
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22589
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11132
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10461
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19145
9	judge	ollama_remote	glm4:latest	0	0	41160

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 135140 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/db0c911f4716.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/db0c911f4716.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/db0c911f4716.tar.gz` (if generated)